ATLAS Master Flightline Clock — Project Handover Dossier

> **Document Purpose:** This is a comprehensive handover document for any future AI assistant working on the ATLAS project. It contains everything you need to pick up exactly where we left off.
> **Last Updated:** July 26, 2026
> **System Version:** v1.6.0 (working, running on user's Mac)

---

1. Project Identity & Purpose

What Is ATLAS?

ATLAS (no official acronym expansion — it's the project codename) is a **specialized 6-row digital display system** designed to manage and synchronize race timing, pilot insertion windows, and heat data during competitive aviation events (specifically **F3K model glider competitions**).

It syncs with the local PC clock for real-time accuracy and guides pilots through two distinct phases:
1. **Pre-Race Phase** — Countdown to track opening, displaying upcoming heat info
2. **Active-Race Phase** — Live timing showing entry windows and heat countdowns

User Context

- **User:** Peter ("petehep"), based in **Adelaide, South Australia**
- **Competition context:** F3K model glider events — multiple heats per day, each with defined track-open/track-close windows and thermalling directions
- **Target deployment:** Standalone "appliance" at a flying field — the display should be visible to pilots from a distance
- **Hardware ambitions:** Physical LED matrix panels (P5/P10) — currently using software simulator
- **Development machine:** MacBook Air (username `petehep`, macOS)
- **Target deployment hardware:** Mac Mini (older/inexpensive model, NOT M4) — or possibly a Windows PC

Project Naming Convention

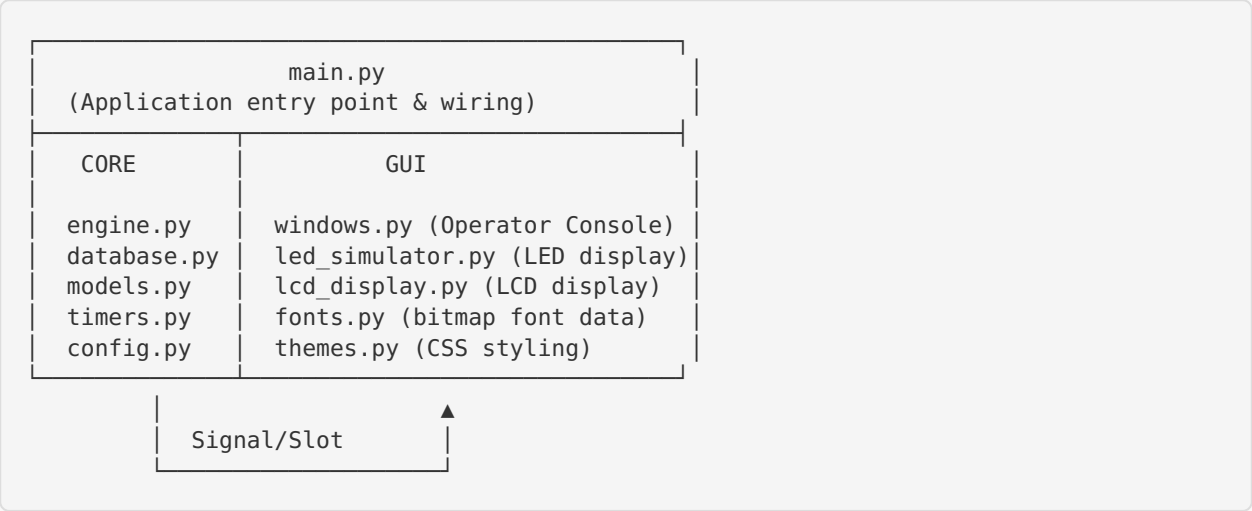
All formal documents follow the pattern: **`ATLAS-XXX-001`**
- `ATLAS-SDS-001` — Software Design Specification
- `ATLAS-ICD-001` — Interface Control Document
- `ATLAS-DEP-001` — Deployment Guide
- `ATLAS-TM-001` — Traceability Matrix (note: current file is named `tm_001_traceability_matrix.pdf`, should be renamed)

---

2. Technical Architecture

Pattern: Model-View-Controller (MVC)

The system uses a clean MVC-style architecture built on **PySide6** (Qt for Python) and **SQLite**.



Signal/Slot Architecture

The engine broadcasts state changes via Qt Signals. Display widgets receive updates without tight coupling:

- **`engine.model_updated`** → Connected to `display.update_model(model)` — fires every 1 second (1 Hz tick)
- **`engine.brightness_changed`** → Connected to `display.set_brightness(value)` — fires when operator adjusts brightness slider

The `AtlasDisplayModel` (Pydantic model) is the data contract between engine and display:

```
class AtlasDisplayModel(BaseModel):
    local_time: str = "00:00:00"
    primary_timer: str = "--:--"
    primary_timer_label: str = "SYSTEM IDLE"
    primary_timer_color: str = "#8888"
    band_color: str = "#3333"
    heat_number: str = "--"
    thermalling_dir: str = "---"
    is_armed: bool = False
    planned_open: str = "--:--"
    planned_close: str = "--:--"
```

Display Backend System

Two display backends exist, selectable from the Settings tab:

1. **`AtlasLEDSimulator`** (`gui/led_simulator.py`) — The original. Emulates discrete LED pixels using a 5×7 bitmap font. Renders rounded rectangles (via `drawEllipse`) to simulate individual LEDs. Supports “full” (128×64) and “compact” (64×64) layouts.

2. **`AtlasLCDDisplay`** (`gui/lcd_display.py`) — **NEW (added this session).** Full-resolution anti-aliased rendering for standard LCD/HDMI monitors. Uses system fonts, scales to window size. Same signal/slot interface as the LED simulator — drop-in replacement.

Both backends implement the same public interface:
- `update_model(model)` — slot for engine state updates
- `set_brightness(value)` — slot for brightness control

Backend selection is handled by a factory function in `main.py` :

```
def _create_display(backend_id, config):
    if backend_id == "lcd":
        return AtlasLCDDisplay(config)
    return AtlasLEDSimulator(config)  # default
```

Engine State Machine

The engine (`core/engine.py`) runs a 1-second tick cycle and manages heat phases:

1. **No heats scheduled** → `is_armed = False` , display shows “WAITING”
2. **Next heat > 15 minutes away** → Shows heat info but stays in WAITING mode
3. **Next heat ≤ 15 minutes away** → Auto-activates the heat (sets status to ‘ACTIVE’)
4. **Active heat, before track_open** → Yellow “TRACK OPENS IN” countdown
5. **Active heat, track open** → Green “ENTER NOW” countdown (pilots can launch)
6. **Active heat, track closed but heat not ended** → Red “HEAT TIME REMAINING” countdown
7. **Heat ended** → Status set to ‘COMPLETED’, cycle repeats

Database Schema

SQLite database at `data/atlas.db` with two tables:

`heats` table:
Column	Type	Notes
id	INTEGER PRIMARY KEY	Auto-increment
heat_number	TEXT	e.g. “01”, “02”
thermalling_dir	TEXT	“LEFT” or “RIGHT”
track_open	INTEGER	Unix timestamp
track_close	INTEGER	Unix timestamp
heat_end	INTEGER	Unix timestamp
status	TEXT	‘READY’, ‘ACTIVE’, ‘COMPLETED’, ‘CANCELLED’

`settings` table:
Column	Type	Notes
key	TEXT PRIMARY KEY	Setting name
value	TEXT	Setting value (all stored as strings)

Known settings keys:
- `competition_name` — Event name string
- `display_width` — “128” or “64”
- `display_height` — “64”
- `led_scale` — Pixel scale factor (default “8”)
- `brightness` — 10–100 integer as string
- `display_backend` — “simulator” or “lcd” (default: “simulator”)
- `auto_boot_enabled` — “0” or “1” (default: “0” = OFF)

---

3. File Structure & Organization

On the User's Mac (ACTUAL structure as reported by user)

```
~/project_atlas/                        ← Project root
├── .git/                               ← Git repository
├── .gitignore
├── .venv/                              ← Python virtual environment
├── archive/                            ← Old/archived files
├── assets/                             ← Static assets
├── atlas_config.json                   ← Legacy JSON config (see note below)
├── atlas.sh                            ← Startup shell script
├── config/                             ← Config directory
├── data/                               ← Contains atlas.db (SQLite database)
│   └── atlas.db
├── docs/                               ← Project documentation (38 items)
├── LED Display Quote.pdf               ← Hardware quote for physical LED panels
├── README.md
├── releases/                           ← Release archives
├── requirements.txt                    ← Python dependencies
├── src/                                ← SOURCE CODE LIVES HERE
│   ├── core/                           ← Business logic
│   │   ├── __init__.py
│   │   ├── config.py                   ← Legacy JSON config loader (not actively used by
main.py)
│   │   ├── database.py                 ← SQLite database manager
│   │   ├── engine.py                   ← 1Hz state machine, heat phase logic
│   │   ├── models.py                   ← Pydantic data models
│   │   └── timers.py                   ← Standalone timer class (legacy, not used by en-
gine)
│   ├── gui/                            ← User interface
│   │   ├── __init__.py
│   │   ├── fonts.py                    ← 5×7 bitmap font definitions (A-Z, 0-9, symbols)
│   │   ├── lcd_display.py              ← NEW: Full-resolution LCD monitor display
│   │   ├── led_simulator.py            ← LED matrix pixel simulator
│   │   ├── themes.py                   ← Qt CSS styling constants
│   │   └── windows.py                  ← Operator Console (Schedule, Live, Settings
tabs)
│   └── main.py                         ← Application entry point
├── tests/                              ← Test directory
└── tools/                              ← Utility scripts
```

CRITICAL: How to Start ATLAS

```
cd ~/project_atlas
source .venv/bin/activate
export PYTHONPATH=$PYTHONPATH:$(pwd)/src
python src/main.py
```

There is also an `atlas.sh` script at the project root (72 bytes) which may contain a shortcut for this.

Note on `atlas_config.json`

This is a **legacy** configuration file from an earlier version. The current `main.py` does NOT use `core/config.py` or `atlas_config.json` — it reads all settings directly from the SQLite database via `core/database.py`. The `AtlasConfig` class in `config.py` and the JSON file are vestigial but harmless.

Note on `core/timers.py`

This is a standalone `AtlasTimerEngine` class from an earlier architecture. The current `engine.py` handles its own timing internally via `QTimer` and `time.time()`. The `timers.py` module is **not imported or used** by any active code. It's vestigial.

---

4. Current System State (as of July 26, 2026)

Working Features

- ✅ **Real-time clock** synced to local system time (1Hz updates)
- ✅ **Heat scheduling** — Add, edit, delete heats with track open/close/end times
- ✅ **Automatic heat activation** — Heats auto-activate when within 15 minutes of track open
- ✅ **Phase-based countdown display** with colour coding (Yellow → Green → Red)
- ✅ **6-row LED simulator** — Full (128×64) and Compact (64×64) layouts
- ✅ **LCD monitor display** — Full-resolution alternative backend (NEW)
- ✅ **Operator Console** with three tabs: LIVE, SCHEDULE, SETTINGS
- ✅ **Brightness control** — Slider with live preview (Night/Overcast/Normal/Bright Day/Full Sun)
- ✅ **Display backend selection** — Dropdown to choose LED Simulator or LCD Monitor
- ✅ **Auto-boot toggle** — Checkbox in Settings (defaults to OFF)
- ✅ **Exit to macOS** button with confirmation dialog
- ✅ **Restart ATLAS** button with confirmation dialog
- ✅ **Abort/Cancel active heat** button on LIVE tab
- ✅ **Settings persistence** — All settings saved to SQLite, survive restarts
- ✅ **Compact mode** — Automatic label abbreviation for small displays

What Launches on Startup (Default — auto_boot OFF)

1. **Operator Console** window (900×850) — The control panel
2. **LED Simulator** window (or LCD Display, depending on settings) — The public-facing clock display

UI Layout — Operator Console

Three tabs navigated by top button bar:

LIVE Tab:
- Large status display showing current heat info or "WAITING"
- "ABORT / CANCEL ACTIVE HEAT" button (red, 80px tall)

SCHEDULE Tab:
- Form to add/edit heats: Heat Number, Thermalling Direction, Track Opens, Track Closes, Heat Ends
- "ADD NEW HEAT" / "UPDATE SELECTED HEAT" button
- "NEW / CLEAR" button
- "DELETE" button (visible only when a row is selected)
- Schedule table showing all heats with columns: ID (hidden), Heat, Dir, Open, Close, Status

SETTINGS Tab:
- Competition Name (text input)
- Display Backend dropdown: "LED Simulator (Software)" / "LCD Monitor (Full Resolution)"
- Display Size dropdown: "128x64 (Large)" / "64x64 (Compact/Half)"
- Sim LED Scale (text input, default "8")

- Display Intensity label + slider (10–100%)
- Auto-Boot to Clock checkbox (default: unchecked)
- “SAVE & APPLY” button (blue, master arm style)
- System Controls group (dark red border):
- “RESTART ATLAS” button (grey)
- “EXIT TO MACOS” button (red)

---

5. Changes Made During This Session

Files Modified (in the workspace at `/home/ubuntu/Uploads/`)

WARNING: These changes exist in the workspace copy. The user’s Mac Mini still has the ORIGINAL files. Changes need to be manually transferred.

1. **`main.py`** — Rewritten to support display backend selection and auto-boot mode
   - Added `DisplayConfig` class to read display settings from database
   - Added `_create_display()` factory function for backend selection
   - Added auto-boot logic (if enabled, only shows display window; otherwise shows both windows)
   - Now imports `AtlasLCDDisplay` alongside `AtlasLEDSimulator`

2. **`core/database.py`** — Added appliance-mode settings
   - Bootstrap now seeds `display_backend` (default: “simulator”) and `auto_boot_enabled` (default: “0”) using `INSERT OR IGNORE` for backward compatibility
   - Added `get_bool_setting()` method that coerces string values to boolean

3. **`gui/windows.py`** — Settings tab enhancements
   - Added “Display Backend” dropdown (`combo_disp_backend`) with LED Simulator / LCD Monitor options
   - Added “Auto-Boot to Clock” checkbox (`chk_auto_boot`)
   - `_save_settings()` now persists backend selection and auto-boot flag
   - Existing Exit and Restart buttons unchanged

4. **`gui/lcd_display.py`** — **NEW FILE**
   - Full-resolution LCD monitor display widget
   - Anti-aliased system fonts, scales to window size
   - Same `update_model` / `set_brightness` interface as LED simulator
   - 6-row layout: Time → Heat Info → Track Window → Phase Label → Big Timer → Status Band

5. **`gui/fonts.py`** — User’s original file, verified correct
   - Contains all A-Z characters including ‘X’ (confirmed present at line 43)
   - The earlier analysis that reported a missing ‘X’ was **incorrect** — the user’s actual file is fine

6. **`core/__init__.py`** and **`gui/__init__.py`** — Created as empty files for Python package imports

Checkbox Visibility Issue

The user reported the auto-boot checkbox wasn’t visible enough. A styling update was applied to make it prominent (golden border, larger text, yellow colour) — **however, the current `windows.py` file on disk shows the ORIGINAL basic styling** (`color: #EEE; font-size: 15px`). The prominent golden styling may need to be re-applied. Here is the intended styling:

```
self.chk_auto_boot.setStyleSheet("""
    QCheckBox {
        color: #FFCC00;
        font-size: 18px;
        font-weight: bold;
        padding: 8px;
        background: #2A2A2A;
        border: 2px solid #FFCC00;
        border-radius: 6px;
    }
    QCheckBox::indicator {
        width: 24px;
        height: 24px;
        border: 2px solid #FFCC00;
        border-radius: 4px;
        background: #1A1A1A;
    }
    QCheckBox::indicator:checked {
        background: #FFCC00;
        border: 2px solid #FF8800;
    }
""")
```

---

6. Development Environment

User's Machine

- **Computer:** MacBook Air (macOS)
- **Username:** `petehep`
- **Home:** `/Users/petehep`
- **Project location:** `~/project_atlas/` (i.e., `/Users/petehep/project_atlas/`)

Python Environment

- **Python version:** 3.14 (bleeding edge — installed via Homebrew or similar)
- **Virtual environment:** `.venv/` in project root
- **Key dependency:** PySide6 6.11.1
- **Other dependencies:** Listed in `requirements.txt` (216 bytes)

How to Activate and Run

```
cd ~/project_atlas
source .venv/bin/activate
export PYTHONPATH=$PYTHONPATH:$(pwd)/src
python src/main.py
```

There May Be a Startup Script

`atlas.sh` exists at the project root (72 bytes). Its contents were not uploaded but it likely contains the activation/run commands above.

---

7. Known Issues & Solutions

Qt “cocoa” Plugin Not Found (SOLVED)

Symptom:

```
qt.qpa.plugin: Could not find the Qt platform plugin "cocoa" in ""
This application failed to start because no Qt platform plugin could be initialized.
zsh: abort      python src/main.py
```

Cause: PySide6 installation became corrupted or misaligned after a macOS/Python update, or after a period of inactivity.

Solution:

```
cd ~/project_atlas
source .venv/bin/activate
pip uninstall PySide6 PySide6-Essentials PySide6-Addons shiboken6 -y
pip install PySide6
```

If that doesn’t work, try force-reinstall:

```
pip install --force-reinstall PySide6
```

If the plugin exists but still fails, set the path explicitly:

```
export QT_QPA_PLATFORM_PLUGIN_PATH=$(python -c "import PySide6, os;
print(os.path.join(os.path.dirname(PySide6.__file__),'Qt','plugins','platforms'))")
python src/main.py
```

Python 3.14 Compatibility

Python 3.14 is very new. If PySide6 wheels aren’t available for 3.14, the user may need to:
- Use `--no-build-isolation` flag
- Or downgrade to Python 3.12/3.13

`pip` Not Found Outside venv

The user’s system doesn’t have `pip` on the global PATH. Always activate `.venv` first before running `pip`.

`PYTHONPATH` Required

The `src/` directory must be on `PYTHONPATH` for imports like `from core.database import ...` to resolve. Without it, Python can’t find the `core` and `gui` packages.

---

8. Documentation Generated

All generated documentation is stored in the AI workspace at `/home/ubuntu/`:

Document	File	Status
Interface Control Document	`ATLAS-ICD-001_Interface_Control_Document.pdf`	✅ Generated this session
Software Design Spec v2.0	`ATLAS-SDS-001-v2.0.pdf`	✅ Generated this session
Deployment Guide v2.0	`ATLAS-DEP-001_Deployment_Guide_v2.0.pdf`	✅ Generated this session
Traceability Matrix	`tm_001_traceability_matrix.pdf`	✅ Generated (needs rename to ATLAS-TM-001)
Deployment Guide v1.0	`ATLAS-DEP-001_Deployment_Guide.pdf`	✅ Original version
Original SDS	`atlas_sds_001_software_design_spec.pdf`	Uploaded by user (original)
Technical Update	`atlas_technical_update_2026-07-02.docx`	Uploaded by user
Handover Dossier	`atlas_system_handover_dossier_2026-07-03.pdf`	Uploaded by user

The user also has a `docs/` directory on their Mac with 38 items — contents not uploaded.

---

9. User Preferences & Working Style

Communication Style

- **Peter is hands-on** — He wants to understand what's happening, not just have things done
- **Don't change code without showing him first** — He explicitly stopped us from making changes until we'd reviewed his actual files
- **He's practical, not academic** — Aviation/engineering background, values things that work
- **Humour is welcome** — The conversation has been casual and collaborative
- **He's in Adelaide** — Australian time zone (ACST/ACDT)

Technical Preferences

- **Documentation naming:** Always use `ATLAS-XXX-001` format
- **Professional aerospace/industrial formatting** for documents

- **Auto-boot defaults to OFF** — System is in active development, he doesn't want it locking into appliance mode yet
- **Ask before modifying** — If files seem missing, ASK the user rather than generating replacements
- **The `src/` directory structure is canonical** — Don't reorganize into a flat layout or different structure
- **Cross-platform matters** — Target is Mac Mini but might be Windows, so keep code platform-agnostic

Things That Went Wrong (Lessons for Future Assistants)

1. **We assumed a flat file structure** and reorganized into `core/` and `gui/` at root level — the user's actual structure has `src/core/` and `src/gui/` under a `src/` directory. Always check first.
2. **We generated a replacement `fonts.py`** when the user already had one — and incorrectly flagged a missing 'X' character that was actually present in the user's file
3. **We made code changes before seeing all the user's files** — User had to explicitly stop us and say "I don't want to change any software until you have a look at all of my current project atlas software files"
4. **The startup command changed** from what worked to what didn't — Always confirm with the user what command they've been using

---

10. Important Context for Continuity

What the User's Mac Has vs What's in the Workspace

The workspace at `/home/ubuntu/Uploads/` contains **modified versions** of the source files (with the new features added). The user's Mac still has the **original** files. To deploy changes, the modified files would need to be downloaded and placed into `~/project_atlas/src/` on the Mac.

Files That Are NEW (don't exist on user's Mac yet)

- `gui/lcd_display.py` — The LCD monitor display backend
- `core/__init__.py` — Package init (may or may not exist on Mac)
- `gui/__init__.py` — Package init (may or may not exist on Mac)

Files That Were MODIFIED (user has original versions)

- `main.py` — Added backend factory and auto-boot logic
- `core/database.py` — Added setting seeds and `get_bool_setting()`
- `gui/windows.py` — Added backend dropdown and auto-boot checkbox to Settings tab

Files That Were NOT CHANGED

- `core/engine.py` — Untouched
- `core/models.py` — Untouched
- `core/config.py` — Untouched (legacy, not actively used)
- `core/timers.py` — Untouched (legacy, not actively used)
- `gui/led_simulator.py` — Untouched
- `gui/fonts.py` — Untouched (user's original is correct)
- `gui/themes.py` — Untouched

The Checkbox Styling

The prominent golden checkbox styling was applied but may have been overwritten. If the user asks about it again, the CSS is documented in Section 5 above.

---

11. Future Work / Roadmap

Near-Term

- [ ] **Transfer modified files to user's Mac** and verify they work
- [ ] **Apply prominent checkbox styling** to auto-boot toggle
- [ ] **Rename** `tm_001_traceability_matrix.pdf` to `ATLAS-TM-001_Traceability_Matrix.pdf`
- [ ] **Test LCD display backend** on user's Mac

Medium-Term

- [ ] **Physical LED hardware integration** — P5 (indoor) or P10 (outdoor) panels via HUB75/HUB75E protocol
- [ ] **macOS LaunchAgent** for true appliance-mode auto-start
- [ ] **Additional display backends** — Artnet DMX, Serial/UART for networked LED controllers
- [ ] **Startup script improvements** — `atlas.sh` could handle venv activation and PYTHONPATH automatically

Long-Term

- [ ] **Dedicated appliance deployment** on Mac Mini at competition field
- [ ] **Remote display** capability (network-driven secondary displays)
- [ ] **Competition day automation** — Pre-loaded schedules, auto-advance through heats

---

12. Quick Reference Card

Start ATLAS

```
cd ~/project_atlas && source .venv/bin/activate && export PYTHONPATH=$PYTHONPATH:$(pwd)/src && python src/main.py
```

Fix Qt Plugin Error

```
cd ~/project_atlas && source .venv/bin/activate && pip install --force-reinstall PySide6
```

Key Files

What	Where
Entry point	`src/main.py`
Engine logic	`src/core/engine.py`
Database	`src/core/database.py`
Data models	`src/core/models.py`
Operator Console	`src/gui/windows.py`
LED Simulator	`src/gui/led_simulator.py`
LCD Display	`src/gui/lcd_display.py`
Bitmap fonts	`src/gui/fonts.py`
Theme/CSS	`src/gui/themes.py`
Database file	`data/atlas.db`

Settings That Require Restart

- Display Backend (LED Simulator ↔ LCD Monitor)
- Display Size (128×64 ↔ 64×64)
- Auto-Boot toggle

Settings That Apply Immediately

- Brightness slider (live update via signal)

End of handover document. If you're a future AI picking this up — read Peter's actual files before changing anything, confirm what command he uses to start the app, and don't assume what's on his Mac matches what's in the workspace.